The Promised Trillion Dollar AI-Coding Market

David Charlot, Open Interface Engineering

Version 1.1, 11 October 2026. HTML: https://research.openie.dev/papers/ai-coding-market/

Contents

Abstract . . . 1
1. Projections and measurements are different objects . . . 2
2. Who projects what . . . 2
3. What the projections assume . . . 5
4. The theory that prices the task . . . 5
5. The measurements . . . 6
6. Where the projections break . . . 6
7. Demand . . . 7
8. Propositions and protocols . . . 7
References . . . 8

Version 1.1, 11 October 2026. Data and code: research.openie.dev/data/. LaTeX PDF: /pdfs/latex/ai-coding-market.pdf (source).

The economics companion to The Reality Code Engine: One Form, Many Fits, and the Low-Entropy Path to Autonomous Coding. The research paper holds the engineering, the theory and the measurements. This essay is where the theory and the economics meet.

Abstract

The trillion-dollar AI coding market is David Charlot's informed hyperbolic claim, and the real projections bear out its scale. A venture estimate puts the value of the world's software developers at about $3 trillion a year and calls AI coding a market that can support "even a trillion-dollar giant" [1]. Two AI coding companies reported over $1 billion and over $2.5 billion in annualized revenue within a year of each other [5, 6]. One frontier lab is valued at $380 billion, and its coding product is its showcase [6]. One provider's token volume grew from 9.7 trillion to over 3.2 quadrillion tokens a month in two years [8]. Capital expenditure by the largest technology companies exceeded $400 billion in 2025 [10].

Every one of these projections assumes the same thing: that the space of autonomous computer automation has one direction. Along it, energy, compute and entropy rise together, a model sits in the loop for every task, and every task pays rent per token. The space has more directions than that. Approaches are placed along continuous axes: energy per task, entropy and determinism, verification strength, reuse and compounding, and model reliance. Many combinations exist, and real systems mix several. The error in the narrative is not any one approach. It is the assumption that the space has only one direction.

The RCE research paper measures one point in that space, chosen to optimize all five axes together. RCE is a native code ecosystem. A program is authored once, from intent, as a Form, a lossless encoding of state, events and pure transitions, and every target language is a faithful projection of it. A state the Form does not declare cannot be written, so a wrong program is refused at entry rather than found and fixed later. Every fit that passes the gate is stored and reused, so the ecosystem compounds. On 10,000 preregistered Forms with a frozen engine and 0 model calls, every Form round trips to identical bytes, every projection is deterministic, every malformed input is refused at entry, and the Python projection replays the reference trace on 9,999 (RCE §2.3). Code from other languages enters by lift, and on lifted code the repair gears restore 520,541 of 520,821 one-token faults and 63.7% of 21,320 single-statement Java

bugs with 0 model calls (RCE §6). Where RCE covers a task, the marginal cost of the task falls toward the energy to run it, and a market priced on generation loses its base on that task. This essay separates the projections from the measurements, names what each projection assumes, and states where it breaks.

Shared thesis, stated once for every paper. Energy to run is the only true metric of computer intelligence. All other factors collapse to zero. The gate form defines what that means. Among runs that pass the task's completeness predicate $C(z)$ within the stated budget and deadline, systems are ranked by joules to completion, J^*. Correctness, deadlines and task size are the gate, not terms in the score. Parameters, tokens, benchmark scores and prices carry zero weight in the ranking (Agency, Section 1). Estimates are never `measured_j`.

Price, a separate claim, for the economics companions. A price trend is a separate claim, labeled as one. Epoch AI reports that the cost of a given level of AI performance has fallen about 47 percent per quarter since 2023, about 13 times per year (Emberson and Roodman, *The plunging price of thought*, 22 September 2026). If that trend continues, energy becomes the dominant marginal cost of inference. That is a cited trend with a condition, not a measured collapse. Spell Check is Global is an existence proof that some computer intelligence commoditizes to bundled zero price. The theory these companions use is in the agency paper, Metered Commit Gates and The Reality Code Engine.

1. Projections and measurements are different objects

Every number below carries one of five labels.

Label	Meaning
Projection	A forecast or a value estimate built on stated assumptions
Company-reported	A figure a company states about itself, such as annualized run-rate revenue (the latest month times twelve) or a funding valuation
Survey estimate	A market-sizing figure from a survey of buyers
Filed	A figure in a regulatory filing
Measured	A count or an energy reading from a run whose artifacts are kept. In RCE, energy carries its own label: `est_j`, `reported_j` or `measured_j`

Only the last row is evidence about the cost of a correct result. The others are evidence about what the market expects.

2. Who projects what

2.1 Developer value: the trillion

Claim	Number	Label	Source
Software developers worldwide	about 30 million (27 million to 47 million by estimate)	Projection input	Appenzeller and Li [1]
Value per developer per year	$100,000 (assumed)	Projection input	[1]
Annual economic contribution	about $3 trillion	Projection	[1]
Developers on GitHub	180 million plus; 518.7 million pull requests merged in the year	Company-reported	GitHub Octoverse 2025 [17]
Value of generative AI across 63 use cases	$2.6 trillion to $4.4 trillion a year; software engineering productivity 20% to 45% of current annual function spend	Projection	McKinsey [15]
Global GDP effect of generative AI	7% over a ten-year period	Projection	Goldman Sachs [16]
Productivity gain from a simple assistant	about 20%	Estimate from interviews	[1]
Productivity gain from a best-of-breed deployment	"at least double"	Estimate "based on anecdotal evidence"	[1]

The developer count is soft: 30 million is a professional estimate, and GitHub alone counts six times that many accounts that build on it [17]. The per-developer value and the productivity multiple are soft too (Section 2.7). McKinsey's 2023 estimate is the original trillion of this market, and it is a value estimate across all functions [15]. The same venture essay says the AI coding ecosystem can "support dozens of billion-dollar companies and even a trillion-dollar giant" [1]. That sentence is the trillion. It is a value estimate. It is not a revenue forecast.

2.2 Tool revenue and enterprise spend

Claim	Number	Label	Source
Enterprise spend on AI coding tools, 2025	$4.0 billion, 55% of departmental AI spend	Survey estimate	Menlo Ventures [2]
Enterprise generative AI spend, 2025	$37 billion	Survey estimate	[2]
AI code tools market, 2026	$9.35 billion, to $29.96 billion by 2031	Projection	Mordor Intelligence [3]

The revenue the market reports today is in the billions. The trillion is the value the market hopes to capture.

2.3 Company revenue and valuations

Company claim	Number	Date	Label	Source
Cursor annualized revenue	over $1 billion	13 November 2025	Company-reported	Cursor [5]
Cursor Pro repriced	from request limits to "$20 of frontier model usage per month at API pricing", with refunds for the change	June to July 2025	Company-reported	Cursor [18]
Cursor Series D	$2.3 billion at $29.3 billion post-money	13 November 2025	Company-reported	[5]
Anthropic run-rate revenue	$14 billion	12 February 2026	Company-reported	Anthropic [6]
Claude Code run-rate revenue	over $2.5 billion	12 February 2026	Company-reported	[6]
Anthropic Series G	$30 billion at $380 billion post-money	12 February 2026	Company-reported	[6]

Anthropic states that the round funds "infrastructure expansions" [6]. Revenue, valuation and compute are one bet.

2.4 Inference, tokens and unit economics

Claim	Number	Label	Source
Google tokens processed per month	9.7 trillion (2024), about 480 trillion (2025), over 3.2 quadrillion (May 2026)	Company-reported	Pichai [8]
Google internal AI developer tools	half a trillion tokens a day in March 2026, more than three trillion a day in May	Company-reported	[8]
Cost of one agentic coding query at list prices	$2.50: a 100,000-token context and 10,000 output tokens at $15 and $75 per million tokens	Illustration	Appenzeller and Li [1]
The same at 3 queries an hour, 7 hours a day, 200 days	about $10,000 a year per developer	Illustration	[1]
Tokens in agentic coding against code reasoning and code chat	about 1,000 times more; runs of one task differ by up to 30 times; higher token use "does not translate into higher accuracy"	Measured (eight models, SWE-bench Verified)	Bai, Huang and Wang [9]
Net losses from Microsoft's investment in OpenAI, quarter ended 30 September 2025	$4.1 billion pre-tax, its equity-method share, in other expense; a $3.1 billion reduction in net income ($0.41 per diluted share)	Filed and company-reported	Microsoft Form 10-Q [7]; Microsoft FY26 Q1 performance [19]

Two lines in the sources state the economics plainly. Pichai: "many companies are already blowing through their annual token budgets, and it's only May" [8]. Appenzeller and Li: "For decades, software development cost was almost purely people's cost, but now LLMs add a substantial opex component" [1].

2.5 Price per token

Claim	Number	Label	Source
Cost of a fixed level of AI performance	down about 47% per quarter, about 13 times per year, since 2023	Measured price series (five benchmarks)	Epoch AI [4]
Price to reach a fixed benchmark score	falls 9 to 900 times a year, depending on the milestone; fastest drops most recent and least certain to persist	Measured price series (log-linear fits)	Epoch AI [34]
Inference cost at equivalent performance	down about 10 times a year: $60 per million tokens in 2021 to $0.06	Estimate	Appenzeller, a16z [20]
DeepSeek API price cut, 29 September 2025	developers' cost "lowered by more than 50%"	Company-reported	DeepSeek [21]

The 13 times a year of [4] is one point in a wide range [34]. The fall is driven by competition and open weights, as the DeepSeek cut shows [21]. It needs no energy accounting. That is the first decline of Section 6. Reuse adds a different one: calls per task.

2.6 Energy and capital

Claim	Number	Label	Source
Data-centre electricity, 2024	about 415 TWh, about 1.5% of world electricity	Estimate	IEA, *Energy and AI* [11]
Data-centre electricity, 2030	about 945 TWh, Base Case	Projection	[11]
Data-centre electricity, 2025 to 2030	485 TWh to 950 TWh, about 3% of world demand	Estimate and projection	IEA, *Key Questions on Energy and AI* [10]
Capex of the largest technology companies	over $400 billion in 2025, expected up another 75% in 2026	Reported and projected	[10]
Alphabet capex	$31 billion in 2022; about $180 to $190 billion expected in 2026	Company-reported and guidance	Pichai [8]
Energy per task	down by at least an order of magnitude a year; reasoning and agentic tasks use "hundreds or thousands of times more energy per query than simple text generation"	Estimate	[10]
One median text prompt to a production assistant	0.24 Wh	Measured by the operator	Elsworth et al. [12]
Stargate	$500 billion over four years for AI infrastructure for one lab, $100 billion deployed at once	Company-announced	OpenAI [22]
Worldwide AI spending, 2025	about $1.5 trillion	Projection	Gartner [24]
Revenue the capex must earn back	"AI's $600B question"	Analysis	Sequoia Capital [23]

The rent that must be serviced is hardware rent. Capex and commitments of this size are paid back through usage priced above its energy. That is the base that reuse removes on covered tasks.

2.7 Productivity evidence

Study	n	Design	Effect	Source
a16z essay	interviews	Estimate	about 20% for a simple assistant; "at least double" best of breed	[1]
GitHub Copilot, HTTP server task	recruited developers	Controlled experiment	55.8% faster on one toy task	Peng et al. [27]
Google enterprise trial	96 engineers	Randomized controlled trial	about 21% less time on task	Paradis et al. [28]

Study	n	Design	Effect	Source
METR, early 2025	16 experienced open-source developers, 246 tasks	Randomized controlled trial	19% slower with AI	Becker et al. [25]
METR, 2026 update	new cohort	Randomized controlled trial	the authors call the new data "an unreliable signal"	METR [26]

The measured effects range from a 19% slowdown to a 56% speed-up on a toy task. Both factors of the value estimate, the developer count and the productivity multiple, are soft. That weakens the trillion. It also weakens any claim that the value reuse frees is a known number.

3. What the projections assume

Read together, the projections rest on four assumptions. Each is a statement about one direction in the design space, not about the value of software.

1. One direction: generation. Every change is written by a model. The value estimate scales productivity by the model, and the cost illustration prices every query [1].
2. High energy and high compute. Token volume, data-centre electricity and capex are projected to grow together [8, 10, 11]. The energy per query falls, and the tokens per task rise faster [9, 10].
3. Per-token rent. Revenue is billed per token or per seat whose cost is tokens. A developer's annual token cost is projected at the scale of a salary [1].
4. AI in the loop forever. Task 10,000 costs what task 1 costs, because nothing the model learned on task 1 runs without the model on task 10,000.

The essay that states the trillion also states its limit: "while substitution pricing works in early markets, eventually the cost of a good converges to its marginal cost, and so does pricing" [1]. This essay agrees, and asks what the marginal cost of a correct coding task is.

4. The theory that prices the task

The RCE research paper supplies three results that turn the question into a quantity.

Correctness is the gate. A task is done when its completeness predicate $C(z)$ holds: its tests pass, a proof exists, or an agreement trace matches (RCE §1). A buyer pays for $C(z) = 1$, not for tokens.

Joules are the ranking. Among systems that pass the gate, the one with the fewest joules to completion, J^*, wins (Agency §1). In a competitive market, sellers that pass the same gate compete on cost, and the cost floor of a task is the energy to complete it plus the price of the proof.

The Form is authored once and reused. A program in the ecosystem is written once, from intent, as a Form, and every target is a projection of it, so the cost of a fit is paid once and every later use runs at the handler floor (RCE §1). Most fits already exist. A program is a tuple $P = (S, E, \delta, D, \lambda)$, and by Gurevich's Abstract State Machine theorem every sequential algorithm has that shape (RCE §3). Programs differ in fit, and most fits exist already: "70% of the code on GitHub consists of clones of previously created files" (RCE §7). Equivalent fits pass the same gate, so extraction picks the least-joule one (RCE Proposition 3).

Proposition E0 (price of a covered task). Let a task be covered when a stored fit passes its gate. Among sellers that pass the gate, competition drives the price of a covered task toward $c_J \cdot J^*_{\text{floor}} + c_{\text{proof}}$, where c_J is the price of a joule, J^*_{floor} is the energy to retrieve, apply and check the fit, and c_{proof} is the cost of the check. Generation enters only on uncovered tasks.

Mixture of Limits is the philosophy behind the proposition: there is a minimal amount of anything to accomplish a task, and any more is waste (Metered Commit Gates, Part II). Per-token rent past the floor bills for waste.

5. The measurements

These are measurements, labeled as RCE labels them. They come from the RCE research paper. The first rows measure the ecosystem itself; the rows after them measure repair of code lifted from other languages (RCE §6).

Measurement	Result	Label	Where
Form ecosystem, 10,000 preregistered Forms, lossless round trip and deterministic projection	10,000 of 10,000, 0 model calls	Measured counts	RCE §2.3
Form ecosystem, malformed inputs and specifications refused at entry	10,000 of 10,000, 0 model calls	Measured counts	RCE §2.3
Form ecosystem, Python projection replays the reference trace byte for byte	9,999 of 10,000, 0 model calls	Measured counts	RCE §2.3
Lifted code, L0 one-token faults, 9 real repositories, held-out test split	520,541 of 520,821 (0.9995, 95% CI 0.9994 to 0.9995), 0 model calls	Measured counts	RCE §6.1
Lifted code, L0, 4 repositories never used for tuning	23,015 of 23,015 (95% CI 0.9998 to 1.0000), 0 model calls	Measured counts	RCE §6.1
Lifted code, L1 Java, ManySStuBs4J 2.0, held-out, whole file	13,586 of 21,320 (0.637, 0.631 to 0.644), 0 model calls; 9,288 of 24,072 (0.386) from the hunk alone	Measured counts	RCE §6.2
Lifted code, L1 Python, TSSB-3M v1.0.0, held-out	27,266 of 157,089 (0.174, 0.172 to 0.175), 0 model calls	Measured counts	RCE §6.2
Energy per solved task, all rungs	0.10 J to 0.40 J	`est_j` (CPU seconds × 10 W)	RCE §6.3
Energy per solved task, RAPL subsample	0.056 J (L0, 3,000 tasks), 0.026 J (L1, 690 solved of 2,000); ±10 J on run totals	`reported_j` (package counter less a stated idle baseline)	RCE §6.3

`measured_j` is reserved for a wall meter and is empty in every row.

6. Where the projections break

Per-token rent breaks on covered tasks. A program authored as a Form is compiled to every target with 0 tokens: 10,000 of 10,000 preregistered Forms compiled, projected and gated with no model call [RCE §2.3]. On lifted code, every covered repair also bought 0 tokens: 520,541 one-token repairs and 40,852 single-statement repairs from real histories [RCE §6.1, §6.2]. A revenue line priced per token earns nothing on a task that needs no token.

AI in the loop forever breaks at promotion. The model is paid once for a new case; the gate admits the fit as a handler, and later cases of the same shape close without it (RCE §4.2). Where that holds, task 10,000 does not cost what task 1 cost. Whether model calls per round reach zero across real specifications is measured at large N by RCE §8.6.

High energy is not required for a correct result. A held-out repair passed the gate at 0.026 to 0.056 reported joules per solved task on a package counter, with no model [RCE §6.3]. One median text prompt to a production assistant is measured at 0.24 Wh, or 864 J [12]. An agentic coding task uses many prompts, and the IEA puts agentic queries at hundreds or thousands of times the energy of a simple one [10]. These are different tasks on different machines, so they are not a ratio. They are an order of magnitude to test, under Protocol E1.

Coverage is measured per class. The rungs give coverage within classes of task: nearly all one-token faults, and 63.7% (Java, whole file; 38.6% from the hunk alone) and 17.4% (Python) of single-statement bugs, where the open cases are the ones whose intent bits lie outside the hunk or are new (RCE §6.0). The share of all real coding volume that falls in covered classes is Proposition E5.

Two declines multiply. The price per call falls 9 to 900 times a year at fixed performance, depending on the task [4, 34]. Promotion drives calls per task to zero on covered cases [RCE §8.6]. The cost per task is the product of the two. A projection that extrapolates token volume prices only the first.

The Spell Check pattern. Spell Check is Global records a capability whose marginal cost fell to about zero and that complemented a tool everyone already had. It was bundled into the editor and its separate price disappeared, as

bundling economics predicts [13, 14]. A covered coding task has the same two properties: its marginal cost falls to the handler floor, and it complements editors and repositories people already have. The prediction is the same: the covered task is bundled, and what stays priced is the uncovered case and the proof.

Bundling by the platform owner. The Spell Check pattern has an owner. The bundle is made by whoever owns the editor, the repository or the cloud. Spolsky named the tactic: commoditize your complement [31]. Nalebuff showed that a bundle can bar entry [32], and Adams and Yellen that it discriminates on price [33]. It is happening: GitHub moved Copilot's individual plans to usage-based billing from 1 June 2026 [30], and Cursor moved Pro from request limits to pass-through API pricing in 2025 [18]. A platform can give the model layer away and keep the rent in the IDE and the cloud. An energy-priced engine competes with a zero-priced bundle only where the gate is open to it: where the tests, the proof and the receipt are the buyer's, not the platform's.

One Form in place of many languages. The RCE paper states the thesis that one universal Form, with every language a projection and every program a lift, replaces the need for any other language (RCE §3.6). The market reading is this. Much of the spend the projections count is translation between languages, frameworks and their versions, and the reconciliation of implementations that drift. In an ecosystem where a program is written once and every target is emitted, that spend has no task to buy. Adoption is the open quantity, and it is a market question, not a theorem.

What survives. The value estimate of section 2.1 can hold. Developers can produce more. What breaks is the claim that the value is captured as rent on generation. The conclusion is conditional. Where coverage is high and the platform does not bundle, value created at a falling marginal cost goes to whoever holds the gate, the proof and the low-entropy floor, and much of it goes to the buyer. Where the platform bundles, the evidence says the gate-holder is the platform.

7. Demand

Cheaper tasks can raise total demand. The venture essay expects it: "we believe Jevon's Paradox in this case holds" [1]. Task Satiation vs. Aggregate Rebound in AI Demand separates the two effects. Per-task appetite stops at the completion predicate. Aggregate use can still grow, because cheaper synthesis opens new tasks. Reuse changes the first effect and leaves the second open. A larger market of covered tasks is a market whose marginal cost is the floor, not the frontier. The one published elasticity is small. Across tasks in Anthropic's API sample, each 1% increase in cost is associated with a 0.29% decrease in usage frequency [29]. That is cross-sectional across tasks, not a price change over time. Read as it stands, demand is inelastic: price declines shrink revenue on a fixed task set, which supports the essay's reading of rent, and they do not by themselves make coding used far more.

8. Propositions and protocols

Proposition E1 (the cost curves cross). For a stream of tasks in one repository, the cumulative cost per resolved task of RCE falls below that of a generation agent after a finite number of tasks. *Protocol:* run RCE Protocol 8.3 and a plain generation agent on the same issue stream under a wall meter. Report `measured_j` and dollars per resolved task by task index, and the crossing index.

Proposition E2 (price follows the floor). Where a coding capability is covered by retrieval and rules, its market price falls toward the energy to run it plus the cost of the proof. *Protocol:* track list prices of a covered capability class against its measured run energy over time, in the manner of the Spell Check price series.

Proposition E3 (compute scale is not a moat on covered tasks). On tasks RCE covers, the resolve rate does not rise with the compute of the last gear. *Protocol:* run RCE Protocol 8.2 with no model, a small local model and a frontier model as the last gear. Compare resolve rates on the tasks closed before the last gear.

Proposition E4 (the token share of revenue). In a repository where coverage grows, the share of task cost paid as tokens falls with coverage. *Protocol:* log tokens, joules and dollars per closed task by round in the RCE convergence protocol with a priced model resolver (RCE §8.6), labeled, and report the token share of cost by round.

Proposition E5 (coverage). The share of real coding task volume that falls in classes RCE covers is measurable. *Protocol:* draw a random sample of merged pull requests from public repositories (GitHub reports 518.7 million merged in a year [17]), classify each by whether frozen G0 to G2 gears produce a gate-passing patch, and report the covered fraction with a 95% interval. Until that number exists, Proposition E0 applies to an unmeasured share.

References

[1] G. Appenzeller and Y. Li, "The Trillion Dollar AI Software Development Stack," 9 October 2025. https://a16z.com/the-trillion-dollar-ai-software-development-stack/

[2] Menlo Ventures, "2025: The State of Generative AI in the Enterprise," December 2025. https://menlovc.com/perspective/2025-the-state-of-generative-ai-in-the-enterprise/

[3] Mordor Intelligence, "AI Code Tools Market Size, Share and 2031 Growth Trends Report," fetched 11 October 2026. https://www.mordorintelligence.com/industry-reports/artificial-intelligence-code-tools-market

[4] B. Emberson and D. Roodman, "The plunging price of thought," Epoch AI, 22 September 2026. https://epoch.ai/publications/the-plunging-price-of-thought

[5] Cursor, "Past, Present, and Future" (Series D announcement), 13 November 2025. https://cursor.com/blog/series-d

[6] Anthropic, "Anthropic raises $30 billion in Series G funding at $380 billion post-money valuation," 12 February 2026. https://www.anthropic.com/news/anthropic-raises-30-billion-series-g-funding-380-billion-post-money-valuation

[7] Microsoft Corporation, Form 10-Q for the quarter ended 30 September 2025, Note 3 and Item 2. https://www.sec.gov/Archives/edgar/data/789019/000119312525256321/msft-20250930.htm

[8] S. Pichai, "I/O 2026: Welcome to the agentic Gemini era," 19 May 2026. https://blog.google/innovation-and-ai/sundar-pichai-io-2026/

[9] L. Bai, Z. Huang and X. Wang, "How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks," 2026. arXiv:2604.22750. https://arxiv.org/abs/2604.22750

[10] International Energy Agency, *Key Questions on Energy and AI*, executive summary, 2026. https://www.iea.org/reports/key-questions-on-energy-and-ai/executive-summary

[11] International Energy Agency, *Energy and AI*, executive summary, April 2025. https://www.iea.org/reports/energy-and-ai/executive-summary

[12] C. Elsworth et al., "Measuring the environmental impact of delivering AI at Google Scale," 2025. arXiv:2508.15734. https://arxiv.org/abs/2508.15734

[13] Y. Bakos and E. Brynjolfsson, "Bundling Information Goods: Pricing, Profits, and Efficiency," *Management Science* 45(12), 1999. doi:10.1287/mnsc.45.12.1613

[14] C. Shapiro and H. R. Varian, *Information Rules: A Strategic Guide to the Network Economy*, Harvard Business School Press, 1998.

[15] McKinsey & Company, "The economic potential of generative AI: The next productivity frontier," June 2023. https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier

[16] Goldman Sachs, "Generative AI could raise global GDP by 7%," April 2023. https://www.goldmansachs.com/intelligence/pages/generative-ai-could-raise-global-gdp-by-7-percent.html

[17] GitHub, "Octoverse: A new developer joins GitHub every second as AI leads TypeScript to #1," 2025. https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/

[18] Cursor, "Clarifying our pricing," June 2025 pricing post. https://cursor.com/blog/june-2025-pricing

[19] Microsoft Investor Relations, FY26 Q1 Performance. https://www.microsoft.com/en-us/investor/earnings/fy-2026-q1/performance

[20] G. Appenzeller, "Welcome to LLMflation: LLM inference cost is going down fast," a16z, November 2024. https://a16z.com/llmflation-llm-inference-cost/

[21] DeepSeek, DeepSeek-V3.2-Exp release and API price reduction, 29 September 2025 (in Chinese). https://api-docs.deepseek.com/zh-cn/news/news250929

[22] OpenAI, “Announcing The Stargate Project,” 21 January 2025. https://openai.com/index/announcing-the-stargate-project/

[23] D. Cahn, “AI’s $600B Question,” Sequoia Capital, 2024. https://www.sequoiacap.com/article/ais-600b-question/

[24] Gartner, “Gartner Says Worldwide AI Spending Will Total $1.5 Trillion in 2025,” 17 September 2025. https://www.gartner.com/en/newsroom/press-releases/2025-09-17-gartner-says-worldwide-ai-spending-will-total-1-point-5-trillion-in-2025

[25] J. Becker, N. Rush, E. Barnes and D. Rein, “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity,” METR, 2025. arXiv:2507.09089. https://arxiv.org/abs/2507.09089

[26] METR, uplift study update, 24 February 2026. https://metr.org/blog/2026-02-24-uplift-update/

[27] S. Peng, E. Kalliamvakou, P. Cihon and M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” 2023. arXiv:2302.06590. https://arxiv.org/abs/2302.06590

[28] E. Paradis, K. Grey, Q. Madison et al., “How much does AI impact development speed? An enterprise-based randomized controlled trial,” 2024. arXiv:2410.12944. https://arxiv.org/abs/2410.12944

[29] Anthropic, “Anthropic Economic Index report: Uneven geographic and enterprise AI adoption,” September 2025. https://www.anthropic.com/research/anthropic-economic-index-september-2025-report

[30] GitHub Japan blog, “GitHub Copilot の個人向けプラン: Pro と Pro+ へのフレックス枠追加と新しい Max プランのご紹介,” 15 May 2026 (in Japanese). https://github.blog/jp/2026-05-15-github-copilot-individual-plans-introducing-flex-allotments-in-pro-and-pro-and-a-new-max-plan/

[31] J. Spolsky, “Strategy Letter V,” Joel on Software, 12 June 2002. https://www.joelonsoftware.com/2002/06/12/strategy-letter-v/

[32] B. Nalebuff, “Bundling as an Entry Barrier,” *Quarterly Journal of Economics* 119(1), 2004. doi:10.1162/003355304772839551

[33] W. J. Adams and J. L. Yellen, “Commodity Bundling and the Burden of Monopoly,” *Quarterly Journal of Economics* 90(3), 1976. doi:10.2307/1886045

[34] Epoch AI, “LLM inference prices have fallen rapidly but unequally across tasks,” data insight. https://epoch.ai/data-insights/llm-inference-price-trends